

Supplementary Notes for “quantmsdiann: a scalable SDRF-driven DIA-NN workflow for reanalysis of single-cell, spatial, and bulk proteomics datasets”

Contents

Supplementary Note 1	Datasets benchmarked and reanalysed	1
Supplementary Note 2	Building DIA-NN container images	3
Supplementary Note 3	Identification filtering and protein counting	3
Supplementary Note 4	Per-process runtime and resource envelopes	4
Supplementary Note 5	ProteoBench protein-group depth across DIA-NN versions	6
Supplementary Note 6	Per-cohort completeness and overlap (cell-line reanalyses)	6
Supplementary Note 7	Software and resources	11

Supplementary Note 1. Datasets benchmarked and reanalysed

Table S1 lists every public dataset used in this study. For each dataset, an SDRF was prepared from the original sample metadata and is provided alongside this supplementary deposition. Raw files were downloaded from PRIDE and MassIVE using pridepy [1]. For each cell-line reanalysis cohort, the original published quantification was downloaded from the PRIDE deposition and used as the comparator. The cell-line bulk DIA cohorts were assembled by programmatic search of PRIDE Archive via the PRIDE API [2], filtering for projects annotated as DIA acquisition on human samples with at least one standardised cell line listed in the Cellosaurus reference, downloadable raw files, and SDRF-compatible sample metadata.

Table S1: Datasets benchmarked and reanalysed with `quantmsdiann`. The “Public date” column is the PRIDE/MassIVE public-release year of the deposit, which can predate the associated publication year cited in the main text (e.g. PXD003539 was deposited in 2016 and published as Guo *et al.* 2019; PXD004701 deposited 2016, published as Sun *et al.* 2023).

Accession	Instrument	Type	Public date
<i>ProteoBench benchmarks</i>			
PXD049412	Orbitrap Astral	Single-cell (Module 9)	2024
PXD062685	timsTOF SCP	diaPASEF (Module 5)	2025
PXD070049	ZenoTOF 7600	Mixed-species (Module 10)	2025
Module 7	Orbitrap Astral	Mixed-species (Module 7)	2025
<i>Bulk cell-line panel</i>			
PXD003539	TripleTOF 5600	Cell line (NCI-60)	2016
PXD030304	TripleTOF 6600	Cell line (ProCan)	2022
PXD004701	TripleTOF 5600	Cell line (Sun 2023)	2016
PXD041421	timsTOF (diaPASEF)	Cell line (Wang 2023)	2023
PXD017199	Q Exactive Plus	Cell line (Tognetti 2021)	2021
<i>Single-cell proteomics</i>			
PXD046357	Orbitrap Astral	Single-cell HeLa (Astral)	2024
PXD049412	Orbitrap Astral	Single-cell A549/H460 (Bubis 2025; also Module 9 benchmark)	2024
MSV000093870	Q Exactive	Single-cell oocyte plexDIA (Galatidou 2024)	2024
<i>Phosphoproteomics</i>			
PXD034128	timsTOF Pro	Phospho diaPASEF (Skowronek 2022)	2022
PXD034623	Orbitrap Exploris 480	Phospho directDIA (Galectin-1)	2023
PXD049692	timsTOF HT	Phospho diaPASEF (NK cells)	2024
<i>Spatial proteomics</i>			
PXD064049	timsTOF SCP	Spatial DVP, diaPASEF (MYCN neuroblastoma)	2025
<i>Scalability experiment</i>			
PXD071075	Orbitrap Eclipse	Single-cell	2025

Supplementary Note 2. Building DIA-NN container images

Every `quantmsdiann` step runs in a versioned container (Docker, Singularity, or Apptainer). Most images are pulled automatically from public registries (BioContainers [3] or `ghcr.io/bigbio`); DIA-NN is the exception because its license restricts redistribution.

Only DIA-NN 1.8.1 may be redistributed, so it is shipped as the default and pulled from BioContainers (`docker.io/biocontainers/diann:v1.8.1_cv1`). Releases from 1.9 onward (1.9.2 through 2.5.1, including the 2.5.1-enterprise build) cannot be redistributed and are therefore *not* shipped as images; instead, the companion `quantms-containers` repository (<https://github.com/bigbio/quantms-containers>) provides one build recipe per release.

A user with a valid DIA-NN download builds the image locally from the matching recipe, for example:

```
cd diann-2.5.1 && docker build -t ghcr.io/bigbio/diann:2.5.1 .
```

(the equivalent Singularity/Apptainer `.sif` image can be built from the same recipe). For `quantmsdiann` to select a locally built image automatically, it must be tagged with the prefix `ghcr.io/bigbio/diann:` followed by the release tag (e.g. `ghcr.io/bigbio/diann:2.5.1` or `ghcr.io/bigbio/diann:2.5.1-enterprise`); the workflow then resolves the container for the requested version through the corresponding `diann_vX_Y_Z` profile selected at runtime.

Supplementary Note 3. Identification filtering and protein counting

All `quantmsdiann` identification counts follow a single rule with exactly one admissible q-value filter per quantity and nothing else (no contaminant/target filter, no positive-quantity filter; zero quantities are counted; decoys are dropped). *Per-run* numbers (per cell, per run): protein groups at `PG.Q.Value` ≤ 0.01 only; precursors at `Q.Value` ≤ 0.01 only. *Global* (dataset-total/union) numbers: protein groups at `Lib.PG.Q.Value` ≤ 0.01 only; precursors at `Lib.Q.Value` ≤ 0.01 only. For deposited datasets the “original” count is the deposited analysis result: the row count of the deposited `pg_matrix/pr_matrix` where only a count matrix is available (HeLa, spatial), or a protein count recomputed from the deposited search output for the reanalysis-recovery cohorts (NCI-60, ProCan, Sun; see below). In plexDIA mode the channel is the per-run unit, so per-cell precursors are counted at `Channel.Q.Value` ≤ 0.01 in addition to `Q.Value` ≤ 0.01 . plexDIA protein-group identification counts are not compared across DIA-NN versions, because the channel-level `PG.Q.Value` confidence used here is not defined in pre-2.0 output; for plexDIA we report `quantmsdiann` depth on its own and assess reproducibility against the deposited analysis rather than an identification gain. For the reanalysis-recovery figure, each “original” count is recomputed from the deposited or public-repository data rather than transcribed from the paper, so every bar is reproducible: NCI-60 (PXD003539) and Sun (PXD004701) from the deposited PRIDE OpenSWATH analyses (for NCI-60, proteins with ≥ 2 distinct peptides in `feature_alignment_requant_matrix.tsv`, reproducing the published 4,284; for Sun, proteotypic SwissProt proteins at the OpenSWATH peak-group q-value ≤ 0.01 in `1R_all_id.txt`, reproducing $\sim 6,091$), and ProCan (PXD030304) from the authors’ figshare peptide-count matrix (proteins with ≥ 2 supporting peptides, 6,698, reproducing the published 6,692). The `quantmsdiann` count is computed under its own global rule (`Lib.PG.Q.Value` ≤ 0.01 , with the ≥ 2 -peptide refinement where the original reports a ≥ 2 -peptide headline); no original-study consistency or detection filter is applied to the `quantmsdiann` side.

Supplementary Note 4. Per-process runtime and resource envelopes

These figures decompose the end-to-end wall-clock and resource use reported in main-text Fig. 2b by Nextflow process, derived from `trace.txt` records collected across the ProteoBench benchmark cohorts and the PXD071075 single-cell production sweep.

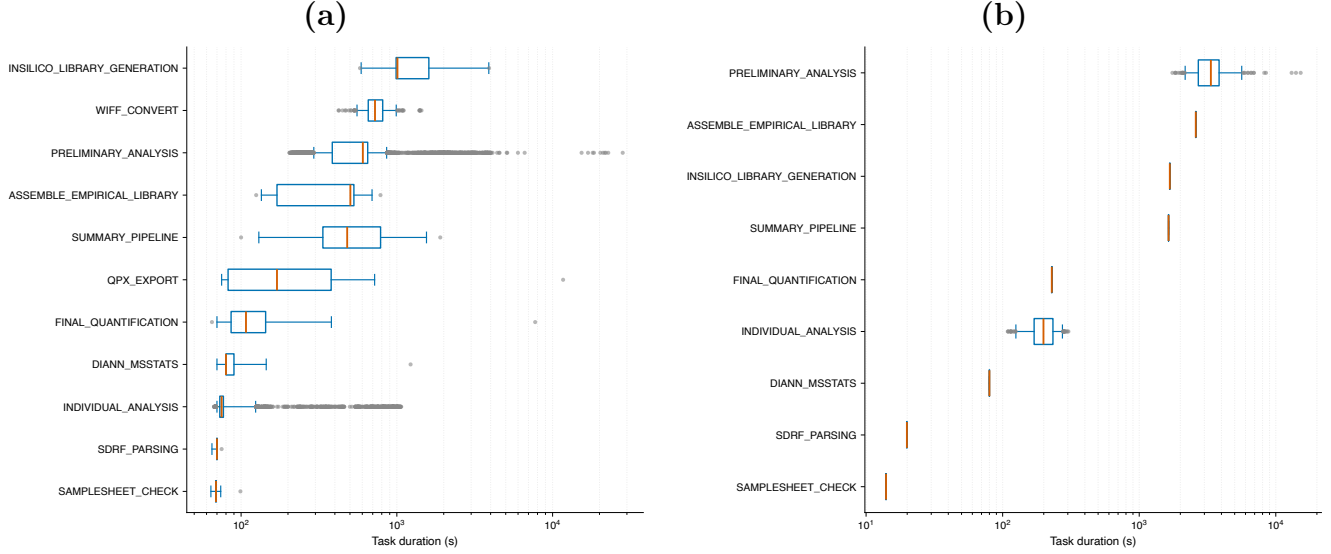


Figure S1: **Per-step wall-clock distribution across Nextflow processes, on the EBI cluster and on an independent third-party cluster.** (a) Distribution of task wall-clock time (boxplot, median and IQR) for each `quantmsdiann` process across the ProteoBench benchmark cohorts and the PXD071075 cluster-node sweep on the EBI HPC cluster. The per-file parallel stages (`FILE_PREPARATION`, `PRELIMINARY_ANALYSIS`, `INDIVIDUAL_ANALYSIS`) run as one task per raw file and determine the workflow's parallelisation envelope, while the collective stages (`INSILICO_LIBRARY_GENERATION`, `ASSEMBLE_EMPIRICAL_LIBRARY`, `FINAL_QUANTIFICATION`, `DIANN_MSSTATS`) run once per cohort and set the wall-clock floor at maximum cluster width. (b) The same per-step breakdown for a `quantmsdiann` run executed independently by a different group on a non-EBI SLURM cluster (Max Delbrück Center, Berlin; 279 raw files, DIA-NN 2.5.0, `quantmsdiann` v2.1.0 under the Singularity profile; total wall-clock 7 h 49 m). The same parallel/collective process structure holds on third-party infrastructure without modification, illustrating the portability of the SDRF-driven, container-pinned workflow.

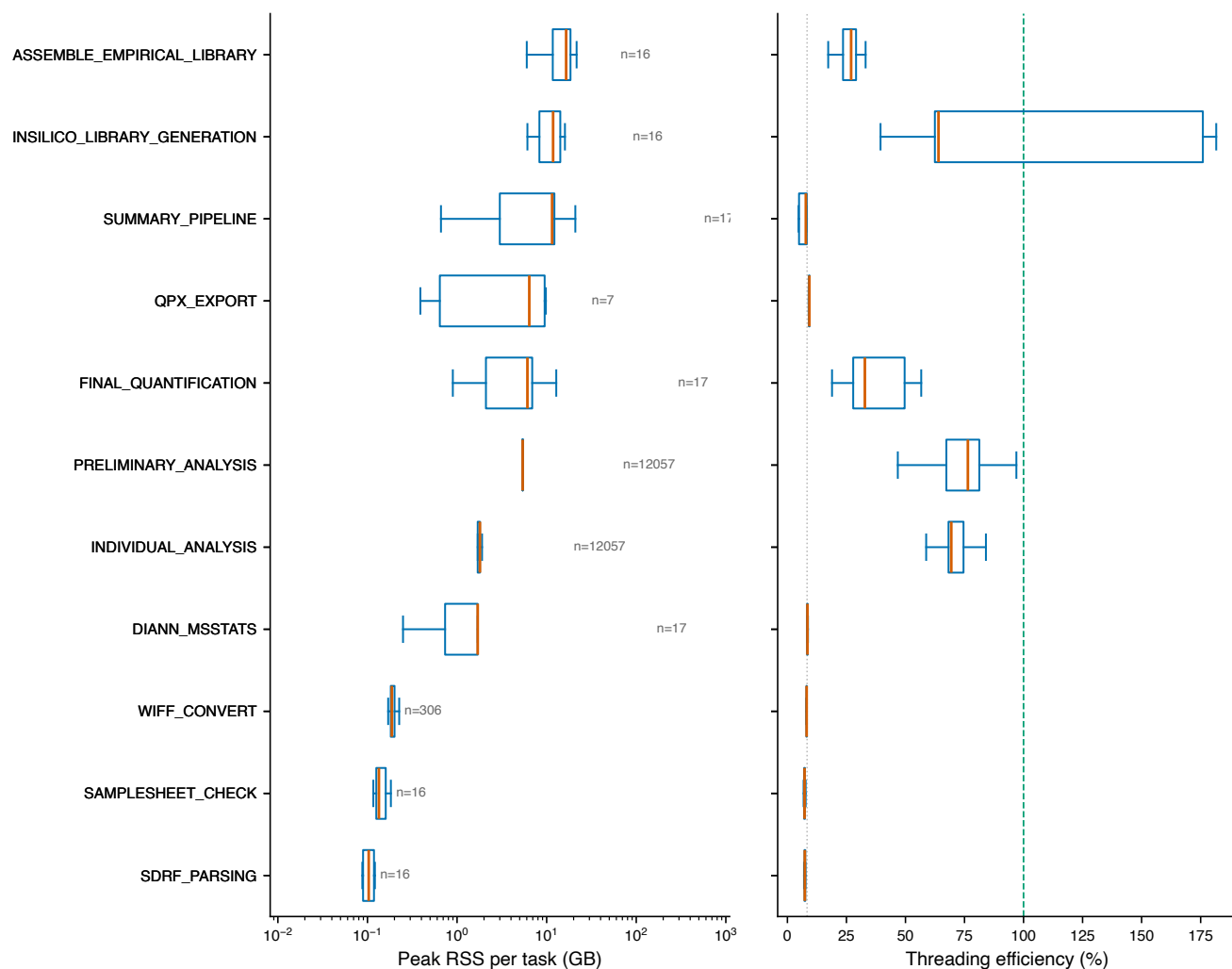


Figure S2: **Peak memory and CPU envelope per Nextflow process.** Per-task peak resident-set-size memory (left panel) and CPU-hours (right panel) per `quantmsdiann` process across the ProteoBench benchmark cohorts and the PXD071075 cluster-node sweep. The collective `FINAL_QUANTIFICATION` stage dominates memory use on large cohorts; per-file stages remain well within the commodity 8–16 GB per-task envelope typical of HPC partitions.

Supplementary Note 5. ProteoBench protein-group depth across DIA-NN versions

The main text reports ProteoBench quantification accuracy (Fig. 2c); the identification depth behind the same runs is summarised here. Total precursors (at the global 1% **Lib.Q.Value**) rise by 9–22% from DIA-NN 1.8.1 to 2.5.1-enterprise on all four modules (largest on timsTOF diaPASEF, 102,528 → 124,756); proteins quantified in every run rise by up to 17% (single-cell Astral, 6,246 → 7,299). Protein groups are counted from the per-precursor DIA-NN report at a 1% run-specific protein-group q-value (**PG.Q.Value**) as the average per run (panel a) and the number quantified in every run (“complete profiles”, panel b).

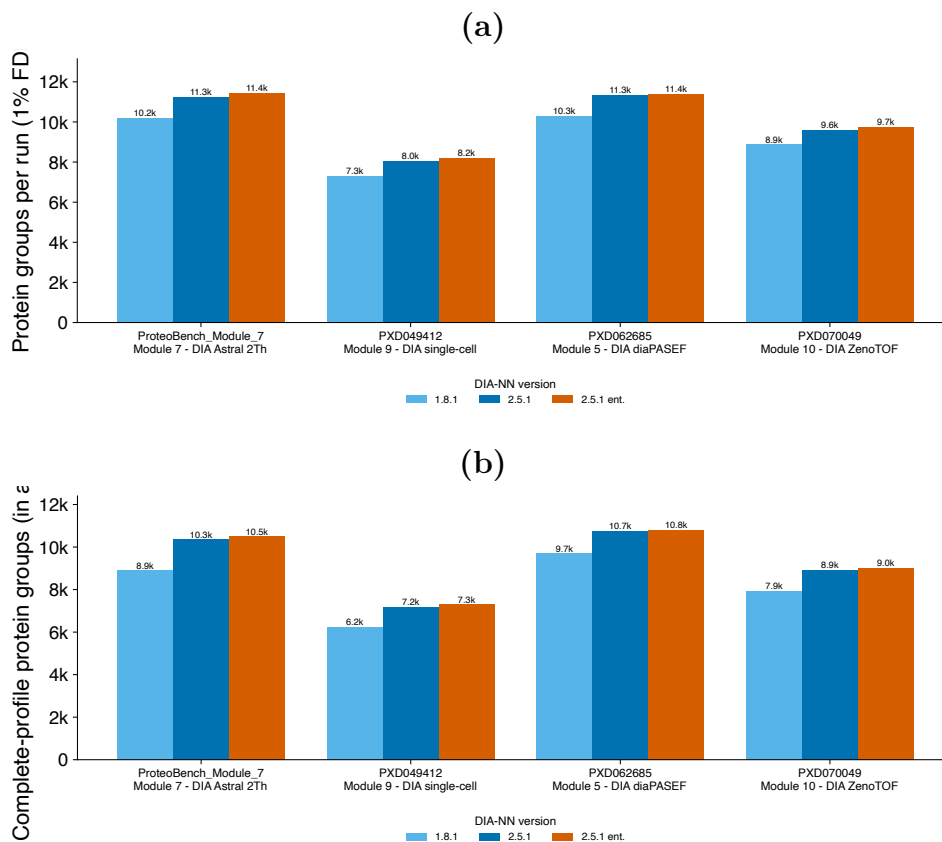


Figure S3: **Protein-group depth on ProteoBench across three DIA-NN configurations and four DIA modalities.** (a) Protein groups per run (averaged over the six replicates) at a 1% run-specific protein-group FDR, by DIA-NN configuration (1.8.1 and 2.5.1 light to dark blue; 2.5.1-enterprise in amber) from one **quantmsdiann** invocation. (b) Complete-profile protein groups (quantified in every run). Both metrics rise monotonically from 1.8.1 to 2.5.1 to the enterprise build on every module; the gain is larger for the complete profiles (up to +18% to the enterprise build) than for the per-run average (+10–12%), reflecting improved cross-run consistency in the newer release.

Supplementary Note 6. Per-cohort completeness and overlap (cell-line reanalyses)

These figures complement main-text Fig. 4 with per-condition completeness, per-protein peptide counts, gene/accession overlap, and per-cell-line missingness for each reanalysed cohort.

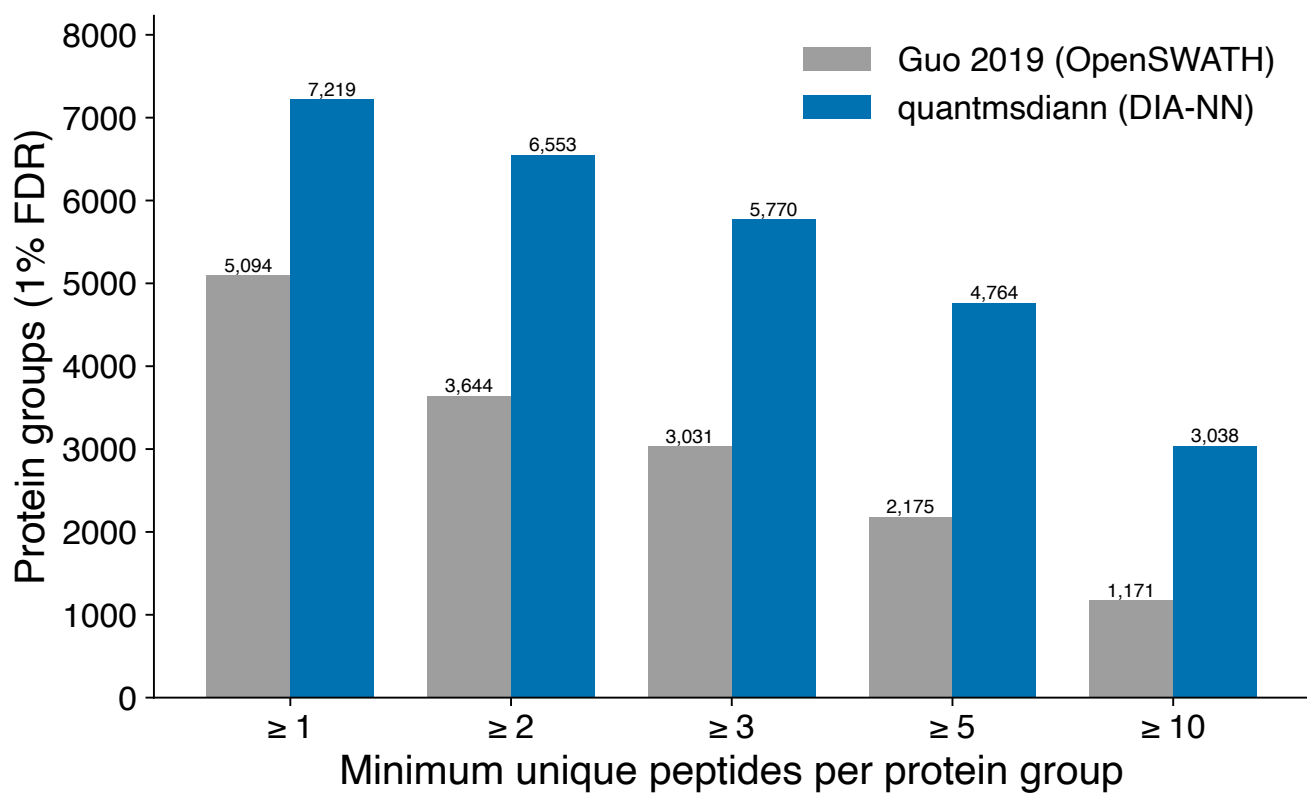


Figure S4: **PXD003539: distribution of peptides per protein group.** quantmsdiann (DIA-NN, blue) versus Guo 2019 OpenSWATH (grey) on the same NCI-60 raw data.

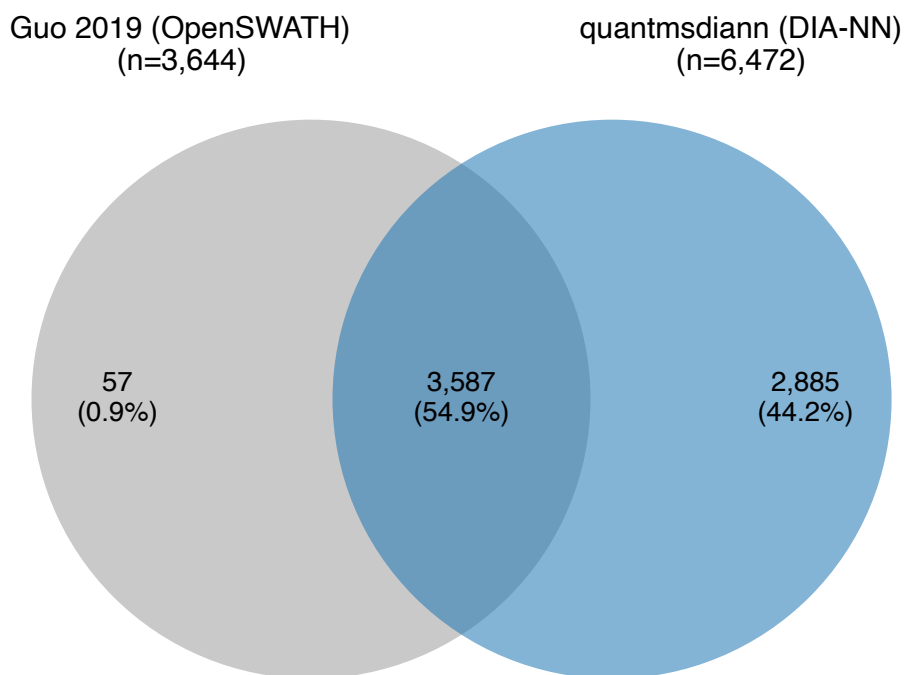


Figure S5: **PXD003539: UniProt-accession overlap between original OpenSWATH analysis and quantmsdiann reanalysis.**

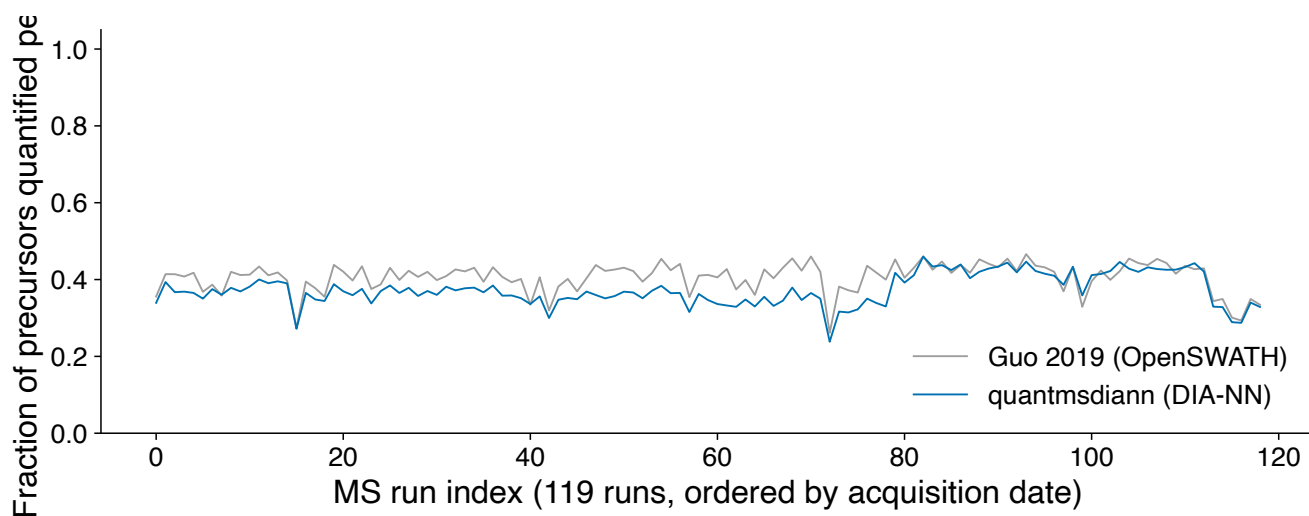


Figure S6: **PXD003539: per-run missing-value fraction.** Distribution of per-run missingness across the NCI-60 panel for both pipelines.

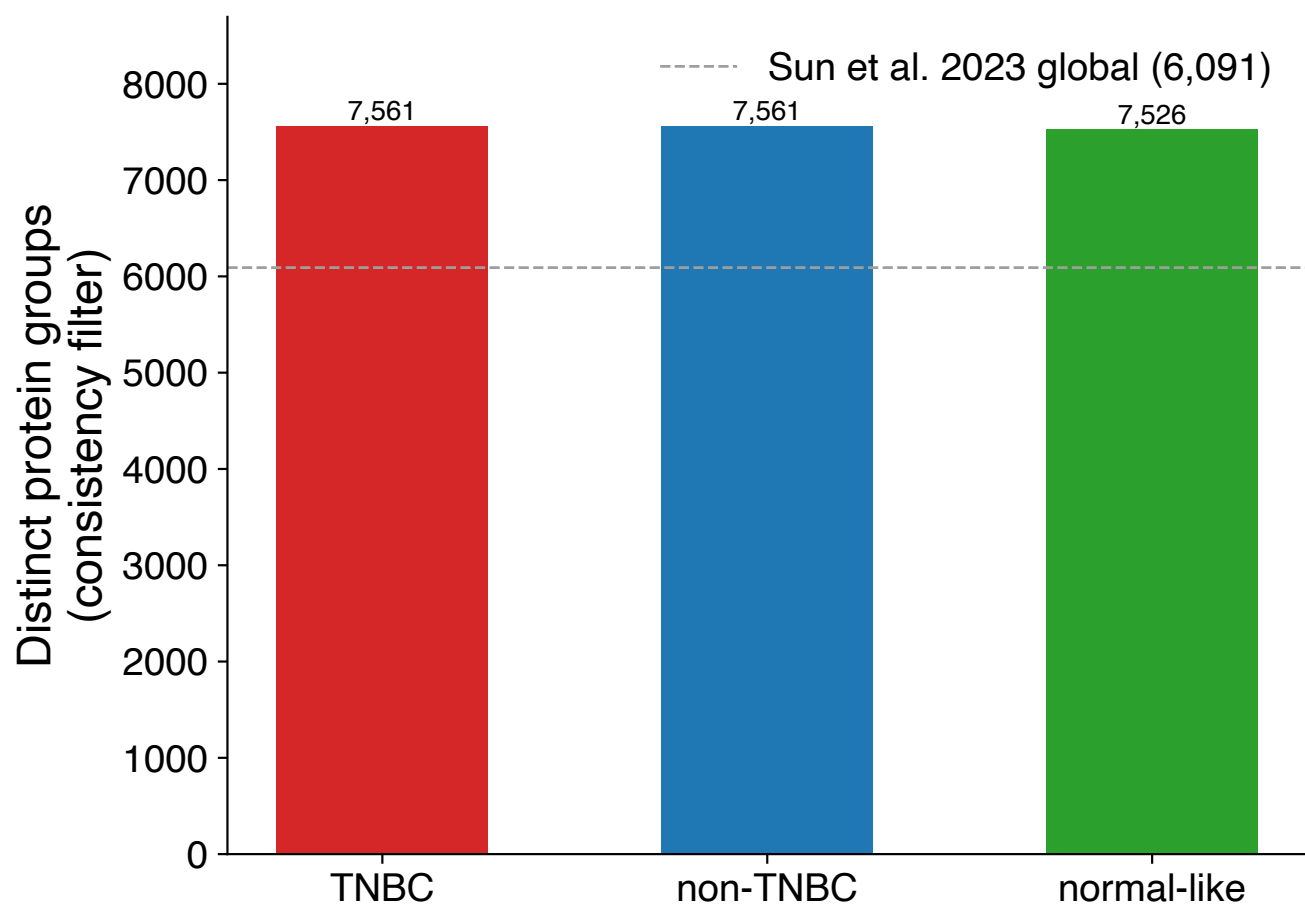


Figure S7: PXD004701 (Sun 2023): protein groups per breast-cancer subtype. Distribution of identifications across the 76 cell lines stratified by TNBC / non-TNBC subtype.

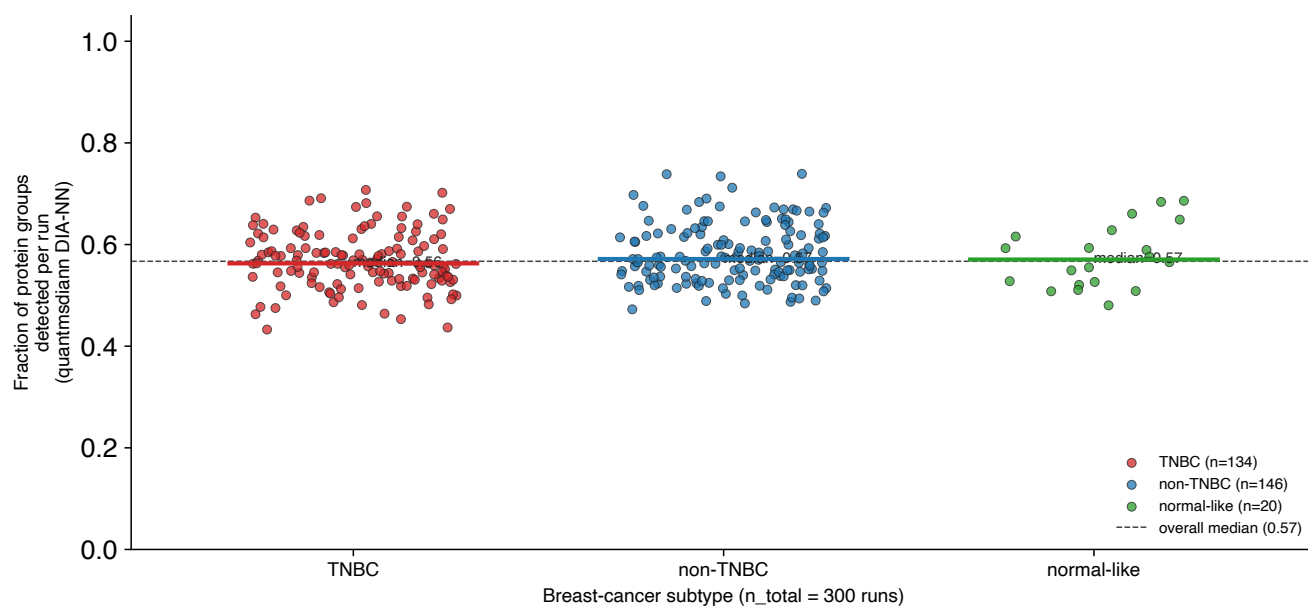


Figure S8: PXD004701 (Sun 2023): per-run missing-value fraction.

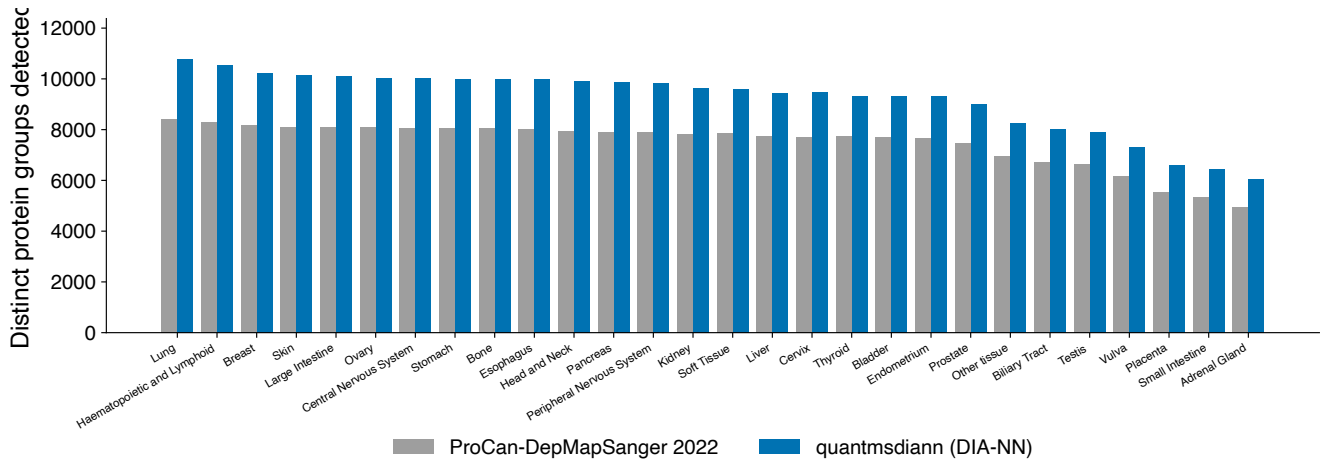


Figure S9: **PXD030304 (ProCan-DepMapSanger): protein groups per tissue.** Distribution of identifications across the 947 cell lines stratified by tissue of origin.

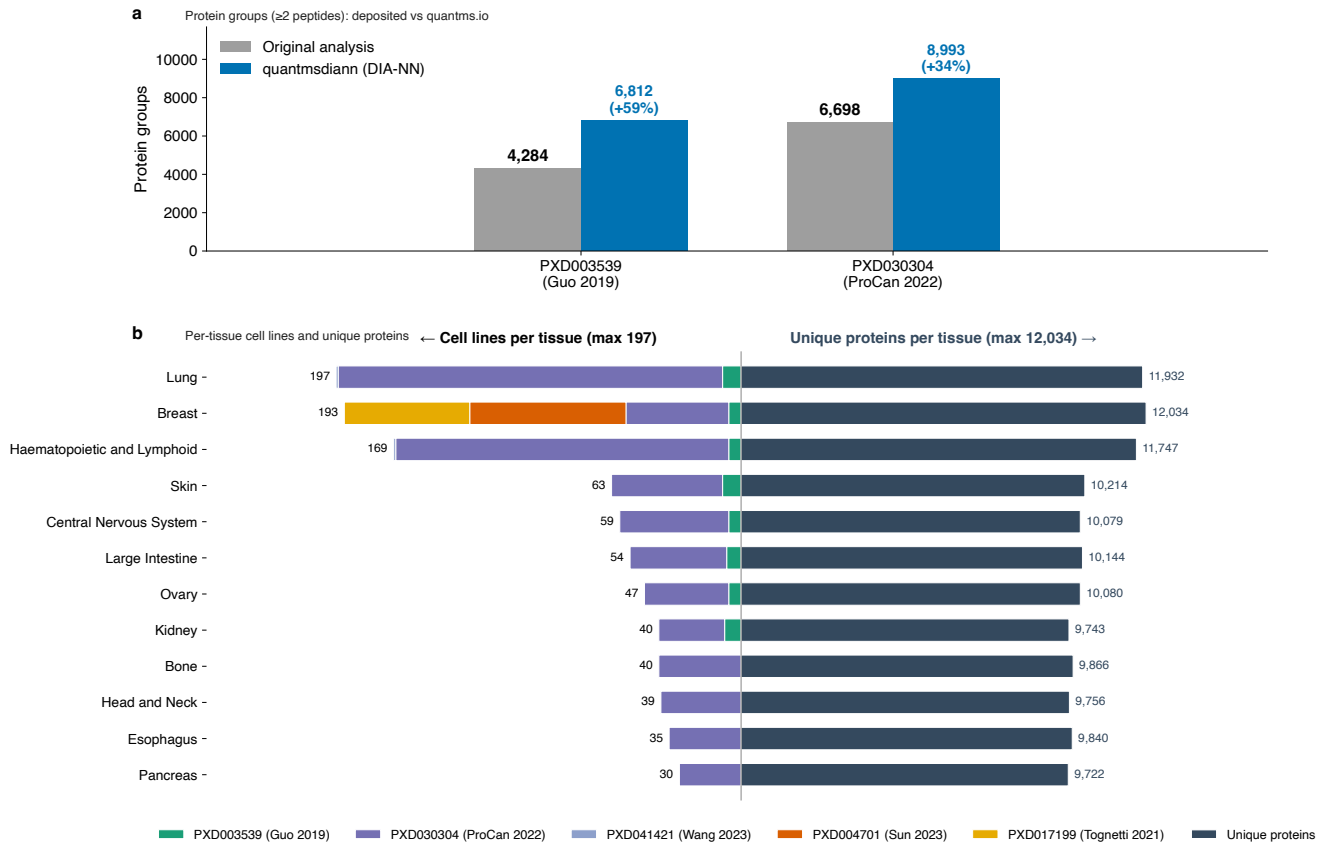


Figure S10: **Pan-cohort of five public DIA cell-line reanalyses by quantmsdiann under a single SDRF-driven invocation.** Combines PXD003539 (NCI-60, Guo 2019), PXD030304 (ProCan-DepMapSanger, 947 lines), PXD004701 (Sun 2023, 76 breast-cancer lines), PXD041421 (Wang 2023), and PXD017199 (Tognetti 2021, 67 breast-cancer lines). (a) For the two cohorts with a published DIA protein-group headline (NCI-60 and ProCan), protein groups requiring ≥ 2 unique peptides: the originally published count (grey) and the quantmsdiann reanalysis (blue), with the same filter applied to both sides (these two comparisons also appear in main-text Fig. 4a). (b) Back-to-back per-tissue summary across the combined panel: cell lines contributing to each tissue, stacked by cohort (left), and the union of unique UniProt proteins detected per tissue (right).

Supplementary Note 7. Software and resources

Table S2 lists the software packages, formats, and public resources used by `quantmsdiann` and in this study, with their canonical repositories or portals. `quantmsdiann` and its supporting tools (`sdrf-pipelines`, `pmultiqc`, `QPX`, `wiffconverter`, container recipes) are released open-source under the `bigbio` organisation; per-version DIA-NN container images are built from the recipes in `quantms-containers` (Supplementary Note 2).

Table S2: Software, formats, and public resources used in this study, with their canonical repositories or portals.

Name	Role in this study	Repository / URL
<i>quantmsdiann and the bigbio ecosystem</i>		
<code>quantmsdiann</code>	SDRF-driven DIA-NN reanalysis workflow (this work)	https://github.com/bigbio/quantmsdiann
<code>quantms</code>	Sibling SDRF-driven multi-engine quantitative proteomics pipeline	https://github.com/bigbio/quantms
<code>quantms-containers</code>	Per-release DIA-NN container recipes	https://github.com/bigbio/quantms-containers
SDRF-Proteomics	Sample-and-Data-Relationship-Format metadata standard	https://github.com/bigbio/proteomics-sample-metadata
<code>sdrf-pipelines</code>	SDRF validation and DIA-NN configuration generation (<code>convert-diann</code>)	https://github.com/bigbio/sdrf-pipelines
<code>qpx</code>	Quantitative Proteomics eXchange format and <code>qpxc</code> CLI	https://github.com/bigbio/qpx
<code>pmultiqc</code>	Proteomics QC reporting (MultiQC extension)	https://github.com/bigbio/pmultiqc
<code>wiffconverter</code>	SCIEX <code>.wiff</code> to mzML conversion	https://github.com/bigbio/wiffconverter
<code>pridepy</code>	PRIDE/MassIVE raw-file download client	https://github.com/PRIDE-Archive/pridepy
<i>Workflow engine and analysis tools</i>		
DIA-NN	Data-independent acquisition search and quantification engine	https://github.com/vdemichev/DiaNN
Nextflow	Workflow orchestration engine	https://github.com/nextflow-io/nextflow
<code>nf-core</code>	Community framework and conventions for the workflow	https://nf-co.re
MultiQC	Aggregated analysis-report framework (extended by <code>pmultiqc</code>)	https://github.com/MultiQC/MultiQC
MSstats	Statistical quantification; target output format	https://github.com/Vitek-Lab/MSstats
ProteoBench	Community benchmarking platform for DIA workflows	https://github.com/Proteobench/ProteoBench
ThermoRawFileParser	Thermo <code>.raw</code> to mzML conversion	https://github.com/compomics/ThermoRawFileParser
<i>Containers and data structures</i>		
Docker	Container runtime (workflow profile)	https://www.docker.com
Apptainer (Singularity)	HPC container runtime (workflow profile)	https://apptainer.org
BioContainers	Source of the default DIA-NN 1.8.1 image	https://biocontainers.pro
DuckDB	In-process engine for querying the QPX Parquet archive	https://github.com/duckdb/duckdb
MuData (<code>.h5mu</code>)	Multimodal container for single-cell export	https://github.com/scverse/mudata

Name	Role in this study	Repository / URL
scverse	Single-cell analysis ecosystem (downstream of MuData export)	https://scverse.org
<i>Data resources and ontologies</i>		
PRIDE Archive	Source repository for deposited datasets	https://www.ebi.ac.uk/pride
ProteomeXchange	Data-deposition consortium / dataset search API	https://www.proteomexchange.org
MassIVE	Source repository for the plexDIA cohort	https://massive.ucsd.edu
UniProt	Reference protein sequences (search FASTA)	https://www.uniprot.org
Cellosaurus	Cell-line nomenclature and tissue annotation	https://www.cellosaurus.org

References

- [1] S. Kamatchinathan, S. Hewapathirana, C. Bandla, S. Insua, J. A. Vizcaíno, Y. Perez-Riverol, pridepy: a Python package to download and search data from PRIDE database, J Open Source Softw 10 (107) (2025) 7563. doi:10.21105/joss.07563.
- [2] E. W. Deutsch, N. Bandeira, Y. Perez-Riverol, V. Sharma, J. J. Carver, L. Mendoza, D. J. Kundu, C. Bandla, S. Kamatchinathan, S. Hewapathirana, et al., The ProteomeXchange consortium in 2026: making proteomics data FAIR, Nucleic Acids Res 54 (D1) (2026) D459–D469. doi:10.1093/nar/gkaf1146.
- [3] F. da Veiga Leprevost, B. A. Grüning, S. Alves Aflitos, H. L. Röst, J. Uszkoreit, H. Barsnes, M. Vaudel, et al., BioContainers: an open-source and community-driven framework for software standardization, Bioinformatics 33 (16) (2017) 2580–2582. doi:10.1093/bioinformatics/btx192.